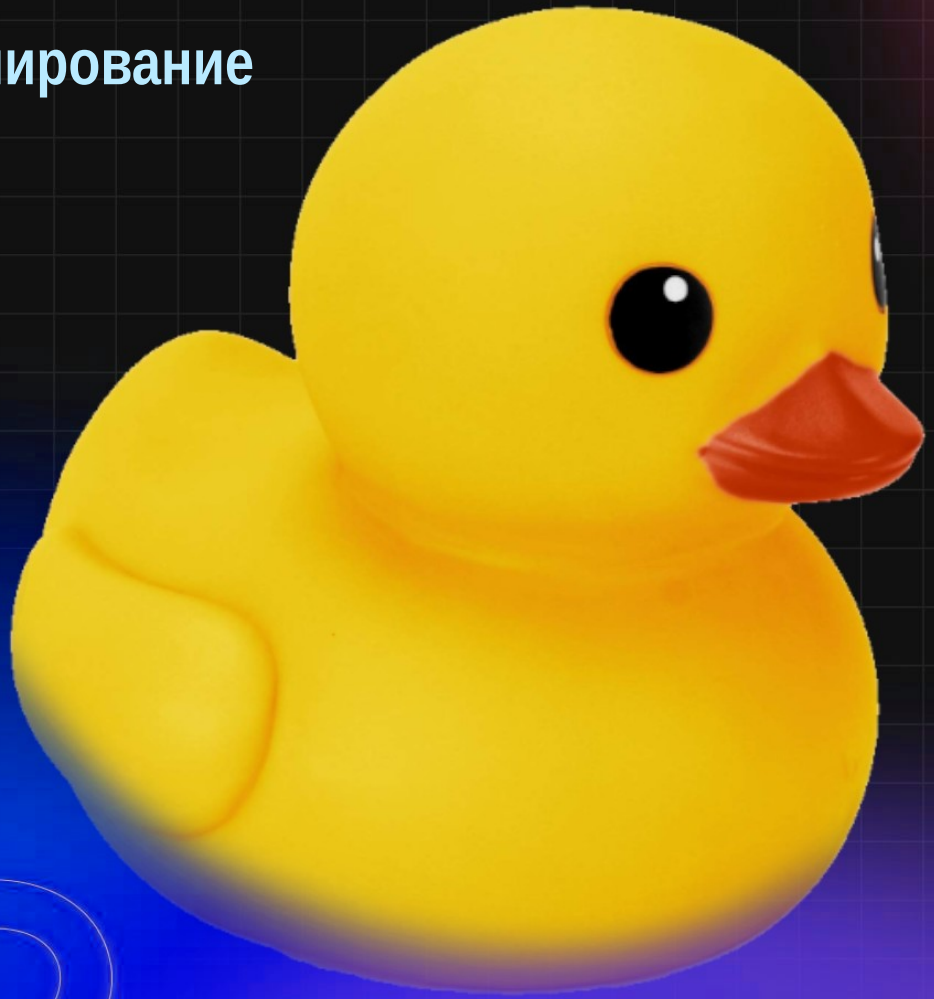
Программирование

2 семестр

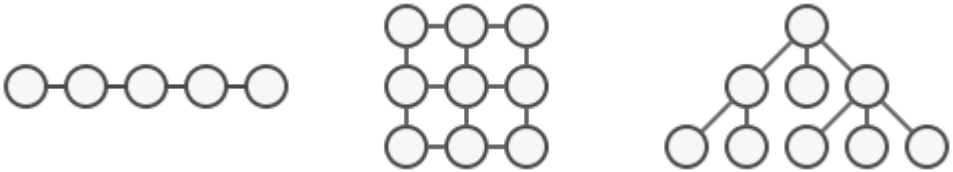
2026

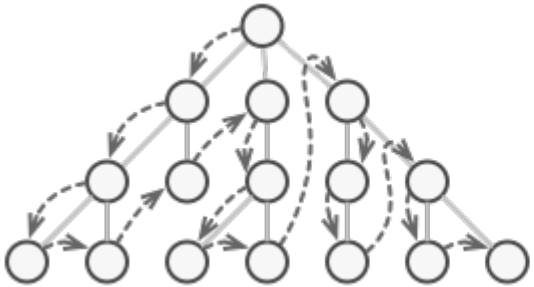


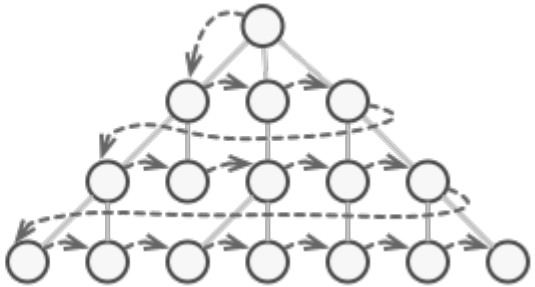
Шаблоны проектирования (продолжение)

Iterator

- Последовательный доступ к элементам коллекции
- Итератор знает внутреннюю структуру коллекции
- Разные способы обхода

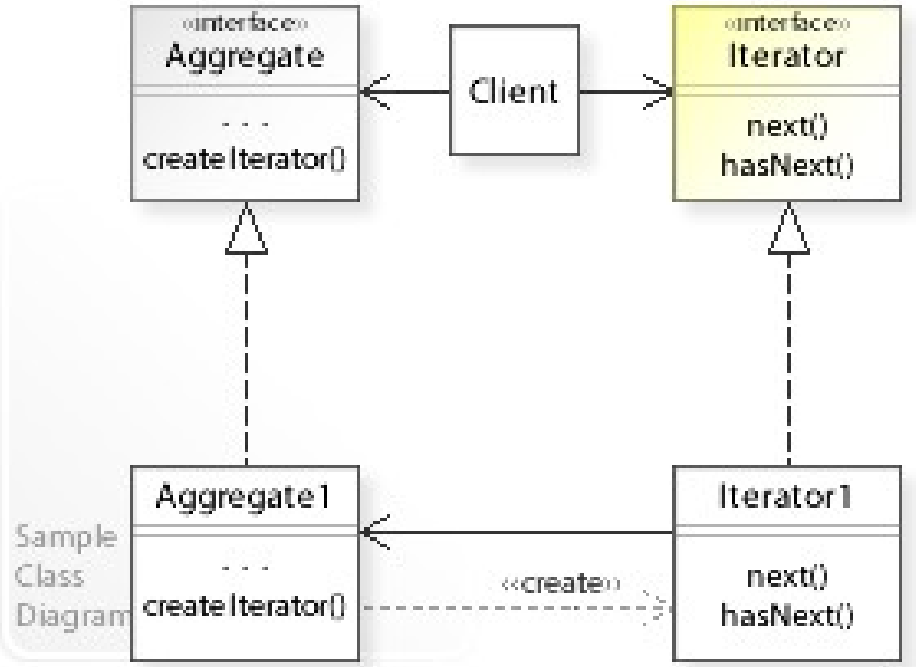






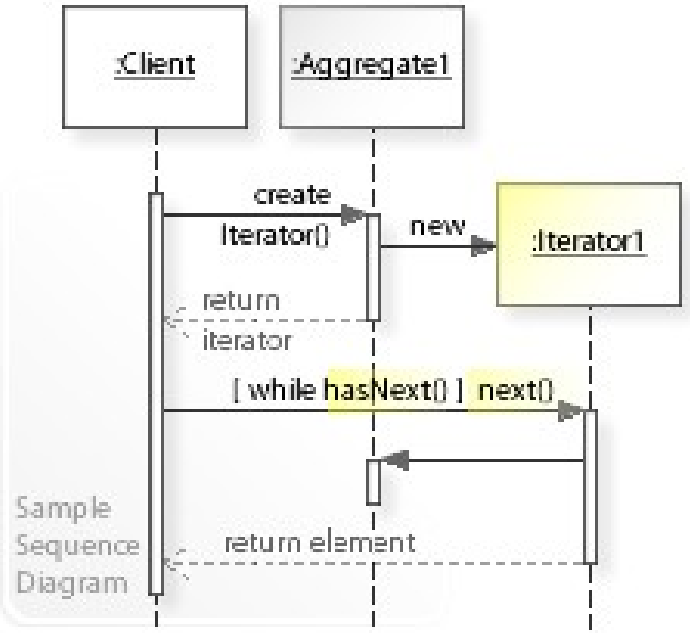
Iterator

- Последовательный доступ к элементам коллекции
- Экскурсия
  - Реальный гид
  - Аудиогид
  - Путеводитель
- Java
  - java.util.Iterator
  - java.util.Scanner



Iterator

- Последовательный доступ к элементам коллекции
- Клиент запрашивает итератор у структуры
- Структура возвращает свой итератор клиенту
- Клиент запрашивает элементы у итератора по очереди



Интерфейс java.lang.Iterable<T>

- Итерируемый объект - **возможен обход** всех элементов
  - Цикл for (x : Iterable)
  - Iterator<T> iterator()

```
Iterable<String> collection = List.of("1","2","3");
for (String str : collection) {
   System.out.println(str);
}

Iterator<String> iterator = collection.iterator();
```

Интерфейс Iterator<E>

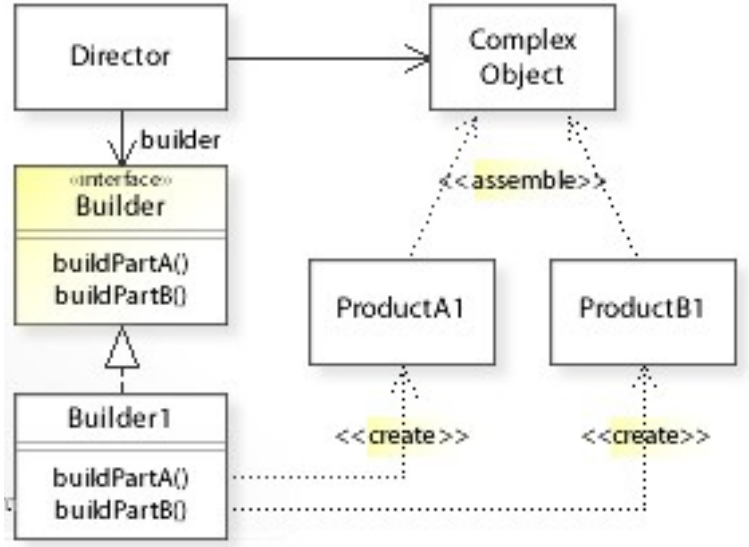
- **Реализует обход** элементов коллекции
  - Позволяет удалять элементы при обходе
  - Методы:
    - boolean hasNext()
    - E next()
    - void remove()

```
Iterator<String> iterator =
      collection.iterator();

while (iterator.hasNext()) {
  String str = iterator.next();
  System.out.println(str);
  if (str == null) {
    iterator.remove();
  }
}
```

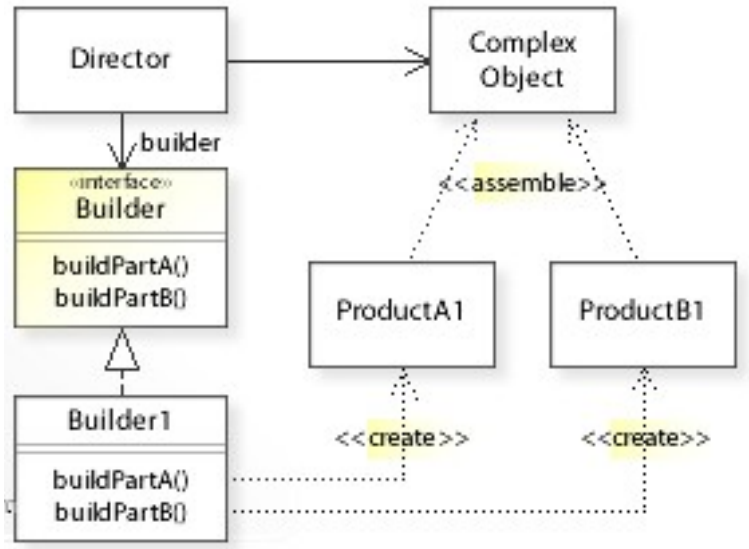
Builder

- Строитель
  - Строит сложный объект по шагам



Builder

- Интерфейс Builder объявляет этапы создания продуктов
- Конкретные строители реализуют этапы создания
- Продукт — создается строителем при вызове метода create()
- Директор управляет строителем для производства нужной конфигурации продукта



Builder

- Класс со множеством полей
- При создании какие-то поля нужно задать
- Возможно не все

```
class Trip {
  private Location[] locations;
  private Transport transport;
  private int numberOfPersons;
  private Date startDate;
  private Date finalDate;
  private Hotel hotel;
  ...
}
```

Builder

- Телескопические конструкторы - ужас №1

```
class Trip {
  public Trip(Date date)
  public Trip(Location location)
  public Trip(Date date, Location location)
  public Trip(Date date, Transport transport)
  public Trip(Date date, Location location, Transport transport)
  public Trip(Date date, Location location, int guests)
  public Trip(Date from, Date until, Location localtion)
  ...
}
```

Builder

- Аргументы по умолчанию - ужас №2

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100,),
activation='relu', *, solver='adam', alpha=0.0001, batch_size='auto',
learning_rate='constant', learning_rate_init=0.001, power_t=0.5,
max_iter=200, shuffle=True, random_state=None, tol=0.0001,
verbose=False, warm_start=False, momentum=0.9, nesterovs_momentum=True,
early_stopping=False, validation_fraction=0.1, beta_1=0.9,
beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000)
```

Builder

- Простой конструктор и набор сеттеров
- Объект не полностью готов в промежуточном состоянии

```
class Trip {
  public Trip()
  public setLocation(Location)
  public setDate(Date)
  public setTransport(Transport)
  ...
}
```

Builder

- Строитель создает объект постепенно
- Методы возвращают строителя
- В конце создаем готовый объект

```
Builder b = new Builder()
   .setWhere("Moscow")
   .setTransport("train")
   .setDate(1,7,2023);
   ...

Trip trip = b.createTrip();
```

- StringBuilder .append()

Builder

- Строитель создает объект постепенно
- Методы возвращают строителя
- В конце создаем готовый объект

```
Builder b = new Builder()
   .setWhere("Moscow")
   .setTransport("train")
   .setDate(1,7,2023);
   ...

Trip trip = b.createTrip();
```

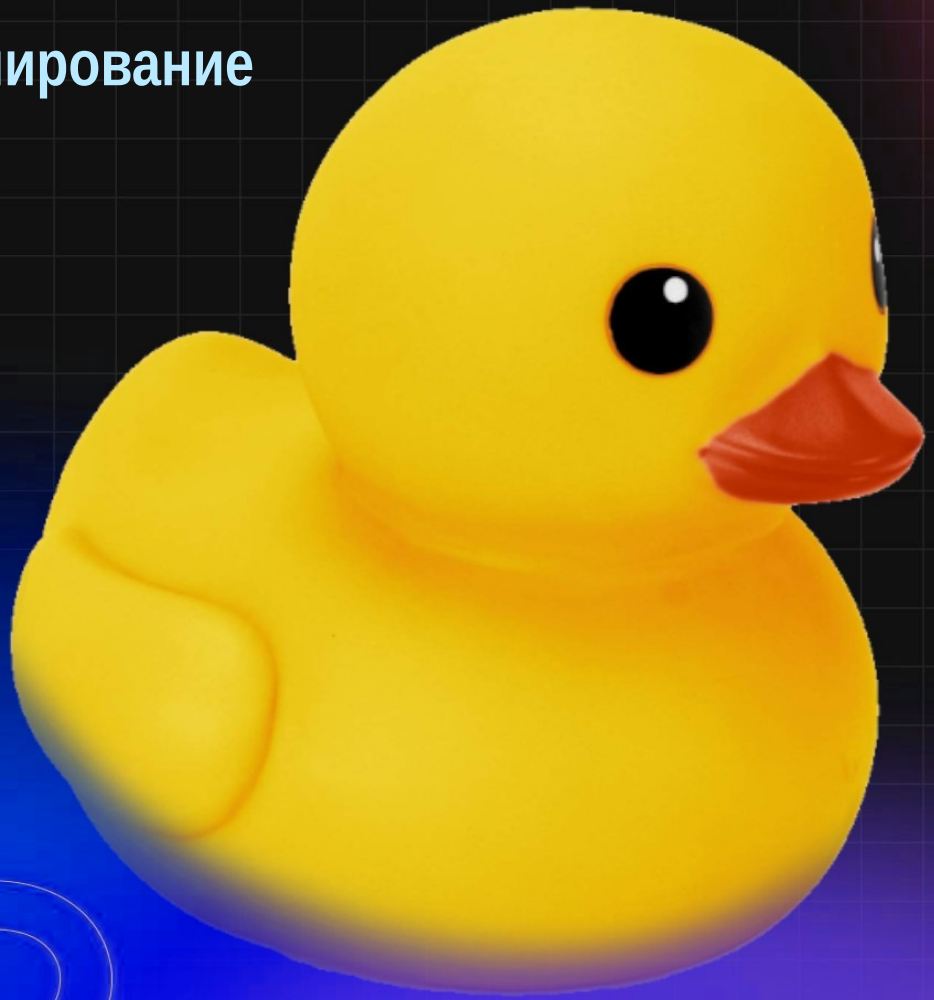
- Директор создает объект с помощью строителя одним методом

```
Director d = new Director();
Trip trip = d.makeTrainTrip("Moscow", new Date(1,7,2023));
```

Программирование

2 семестр

2026



Сравнение элементов

Сравнение элементов

- Интерфейсы Comparable<T> и Comparator<T>
  - основной метод возвращает int (< 0, == 0, > 0)
- java.lang.Comparable<T> (x.compareTo(y))
  - естественный порядок сортировки
  - реализуется при создании класса
  - реализован в большинстве библиотечных классов
- java.util.Comparator<T> (c.compare(x, y))
  - любой необходимый порядок сортировки
  - объект создается по мере необходимости

Обобщенный алгоритм сортировки

1) Выбираем два элемента
2) Как-то сравниваем - Comparable или Comparator
3) При необходимости перемещаем элементы

- Готовые эффективные алгоритмы сортировки
  - List.sort() - stable, adaptive, iterative mergesort
  - Arrays.sort()
- Все, что нужно - предоставить метод сравнения

Сортировка (естественный порядок)

ИТМО

```
public class Test {
  List<String> list;
  public void printSorted() {
    list.sort(null);
    for (String s : list) {
      System.out.println(s);
    }
  }
}
```

```
public class String implements Comparable<String>
```

Java

Сортировка (естественная) - свой класс

ИТМО

```
public class Test {
  List<Person> list;
  public void printSorted() {
    list.sort(null);
    for (Person p : list) {
      System.out.println(s);
    }
  }
}
```

```
public class Person implements Comparable<Person> {
  private String name;
  @Override
  public int compareTo(Person another) {
    return this.name.compareTo(another.name);
  }
}
```

Java

Сортировка (отдельный компаратор)

ИТМО

```
public class Test {
  List<String> list;
  public void printSorted() {
    list.sort(new LengthComparator());
    for (Person p : list) {
      System.out.println(s);
    }
  }
}
```

```
class LengthComparator implements Comparator<String> {
  public int compare(String s1, String s2) {
    return s1.length() - s2.length();
  }
}
```

Java

Сортировка (вложенный static класс)

ИТМО

```
public class Test {
  List<String> list;
  public void printSorted() {
    list.sort(new LengthComparator());
    for (Person p : list) {
      System.out.println(s);
    }
  }
  static class LengthComparator implements Comparator<String> {
    public int compare(String s1, String s2) {
      return s1.length() - s2.length();
    }
  }
}
```

Java

Сортировка (внутренний класс)

ИТМО

```
public class Test {
  List<String> list;
  public void printSorted() {
    list.sort(this.new LengthComparator());
    for (Person p : list) {
      System.out.println(s);
    }
  }
  class LengthComparator implements Comparator<String> {
    public int compare(String s1, String s2) {
      return s1.length() - s2.length();
    }
  }
}
```

Java

Сортировка (локальный класс)

ИТМО

```
public class Test {
  List<String> list;
  public void printSorted() {
    class LengthComparator implements Comparator<String> {
      public int compare(String s1, String s2) {
        return s1.length() - s2.length();
      }
    }
    list.sort(new LengthComparator());
    for (Person p : list) {
      System.out.println(s);
    }
  }
}
```

Java

Сортировка (анонимный класс)

ИТМО

```
public class Test {
  List<String> list;
  public void printSorted() {
    list.sort(new Comparator<String>() {
      public int compare(String s1, String s2) {
        return s1.length() - s2.length();
      }
    });
    for (Person p : list) {
      System.out.println(s);
    }
  }
}
```

Java

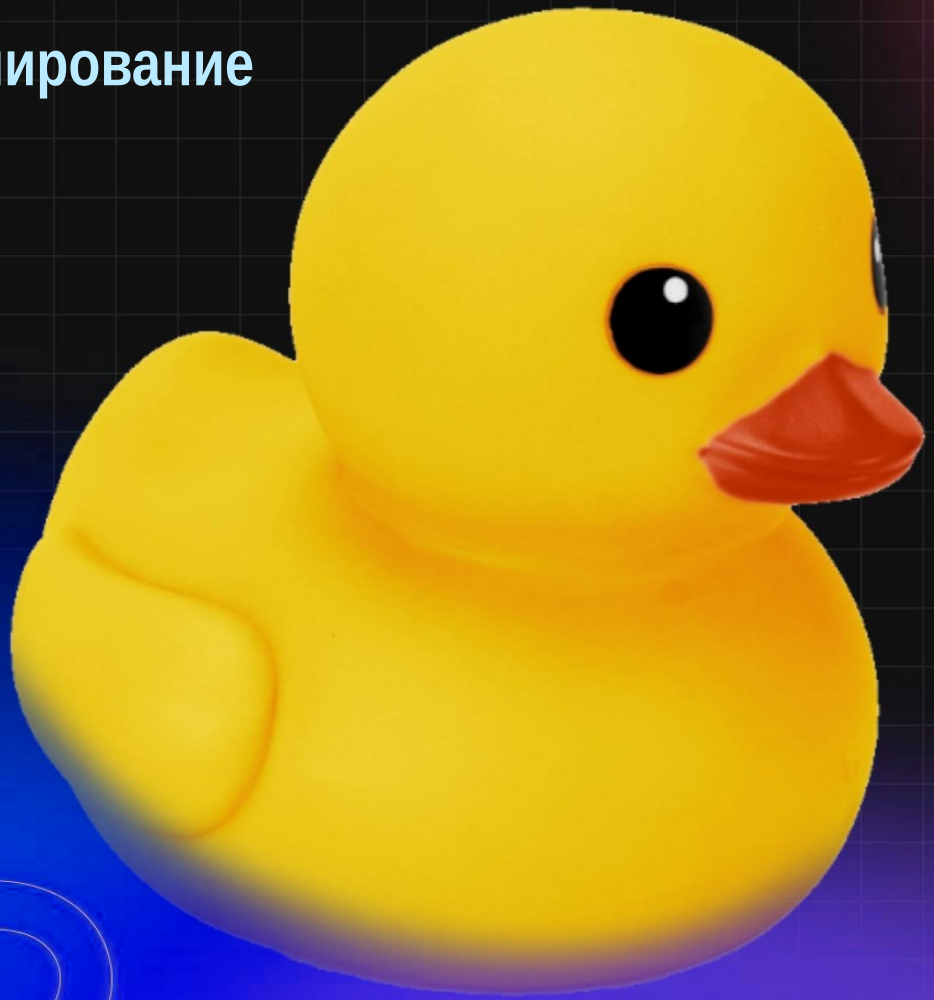
Сортировка (лямбда-выражение)

```
public class Test {
  List<String> list;
  public void printSorted() {
    list.sort((s1, s2) -> s1.length() - s2.length(););
    for (Person p : list) {
      System.out.println(s);
    }
  }
}
```

Программирование

2 семестр

2026

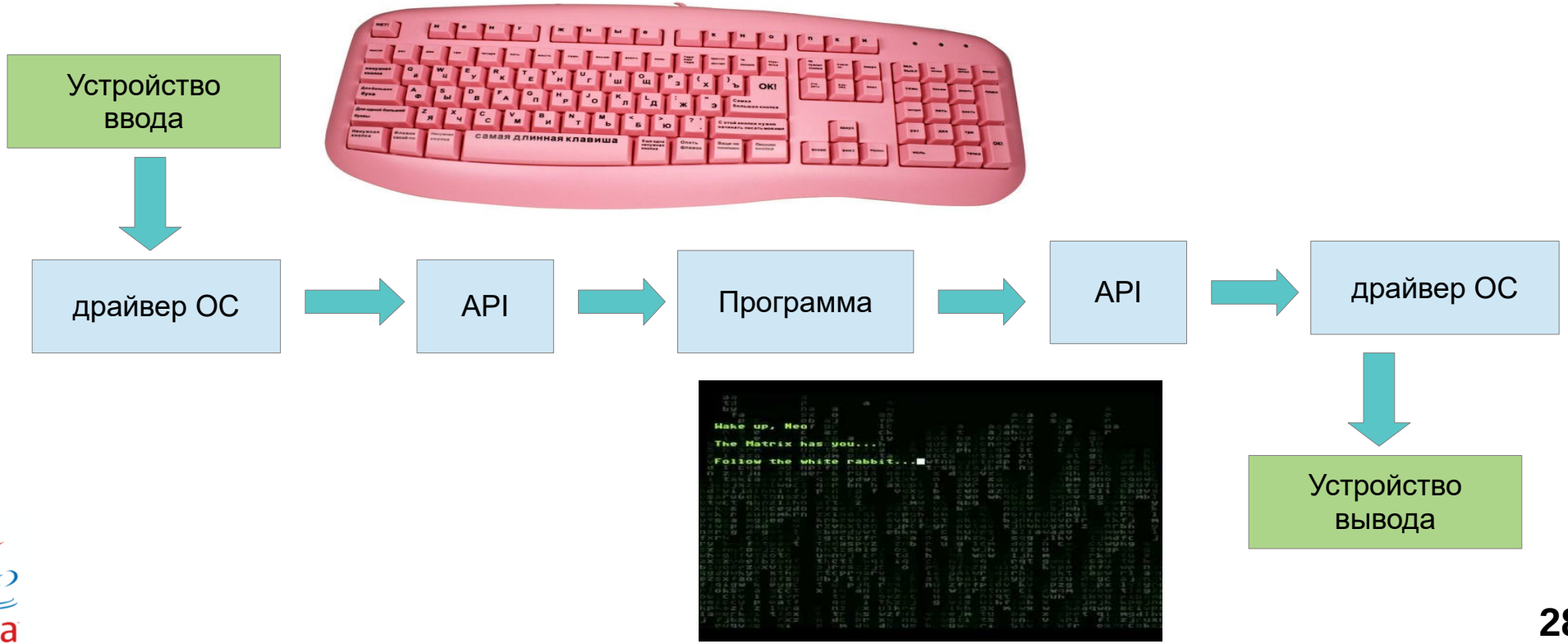


Ввод-вывод

Ввод-вывод в Java

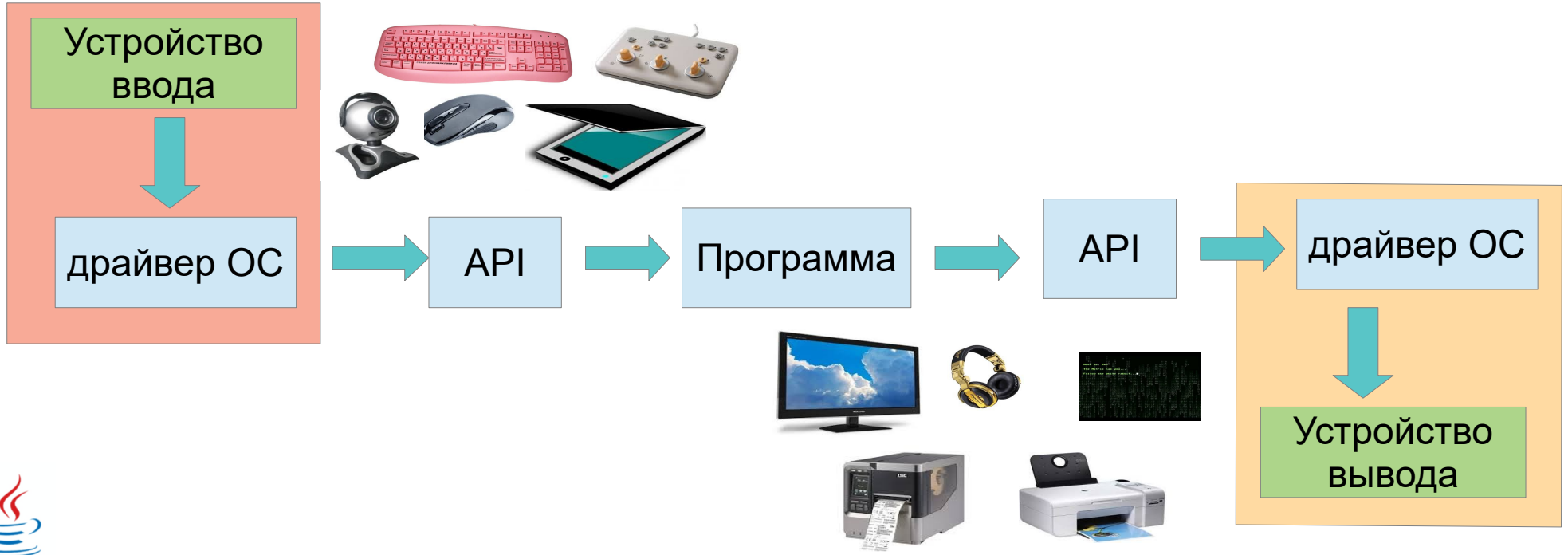
- java.io
  - Абстракция потока ввода-вывода
  - Данные - поток байтов/символов
- java.nio
  - Абстракция канала и буфера
  - Буфер - хранение данных
  - Канал - соединение для передачи данных

Ввод-вывод


Устройство ввода
самая длинная клавиша
драйвер ОС
API
Программа
API
драйвер ОС
Wake up, Neo!
The Matrix has you...
Follow the white rabbit...
Устройство вывода

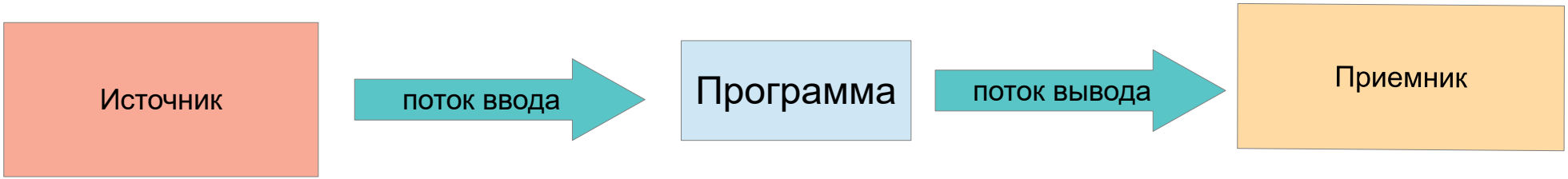
Абстракция ввода-вывода

- Потоки ввода-вывода
- Источник данных и приемник данных



Абстракция ввода-вывода

- Потоки ввода-вывода
- Источник данных и приемник данных



Пакет java.io

- Байтовые и символьные потоки данных (I/O streams)
  - Поток — последовательность данных (байтов, символов, примитивных типов, объектов)
  - Поток ввода — для чтения данных из источника
  - Поток вывода — для записи данных в приемник
- Старый интерфейс работы с файлами — класс File

Представление информации байтами

```
int inum = 51966;


51966₁₀ = 00000000_00000000_11001010_11111110₂

// big-endian                  // little-endian
  00000000                       11111110
  00000000                       11001010
  11001010                       00000000
  11111110                       00000000
```

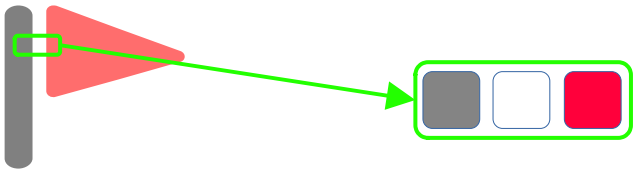
Представление информации байтами

```
int inum = 51966;
float fnum = 51966f;
```

51966_{10} = $00000000_00000000_11001010_11111110_2$

$51966f_{10}$ = $01000111_01001010_11111110_00000000_2$

Представление информации байтами



R,G,B
120,120,120
255,255,255
255,000,000

01111000
01111000
01111000
11111111
11111111
11111111
11111111
00000000
00000000

Представление текста - кодировки

ИТМО

```
Hello!
```

```
72 101 108 108 111 21
```

A	- 65	H	- 72
B	- 66	e	- 101
C	- 67	l	- 108
D	- 68	o	- 111
E	- 69	!	- 21

Java

Представление текста - кодировки

- ASCII - 7 бит, 128 символов,
  - A-Z, a-z, 0-9, !@#$%^...
- 8-битные - 128-255 - разные варианты
  - код символа 'Ё'
  - ISO-8859-1 161
  - KOI8-R 179
  - CP1251 168
  - CP866 240
  - MacCyrillic 221
- Unicode
  - код символа 'Ё' 1025
  - UTF-8
    - 224 144 129
  - UTF-16 + Byte Order Mark
    - 254 255 004 001

Байты и символы

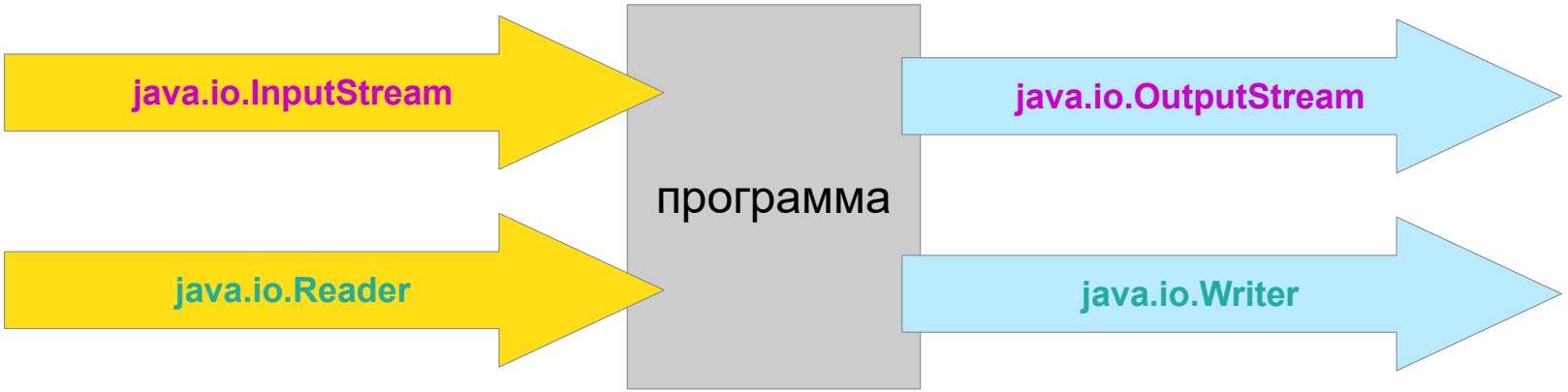
```
Integer number = 51966;

number.toString()      // "51966"
'5' '1' '9' '6' '6'
11111110 11111111 00000000 00110101 00000000 00110001
00000000 00111001 00000000 00110110 00000000 00110110
number.toHexString()  //  "cafe"
'c' 'a' 'f' 'e'
11111110 11111111 00000000 01100011 00000000 01100001
00000000 01100110 00000000 01100101

UTF-16 – BOM (byte order mark)    'FEFF'
```

Потоки ввода-вывода

- Базовые абстрактные классы для потоков
- Ввода и вывода
- Байтовые и символьные



java.io.InputStream

- abstract **int** read()
  - прочитанный байт
  - -1 (конец потока)

00000000	00000000	00000000	10100111
11111111	11111111	11111111	11111111

- int read(byte[] buf, int off, int len)

- int available()
- long skip(long n)
- void close()
- boolean markSupported()
- void mark(int limit)
- void reset()

java.io.Reader

- abstract **int** read(char[] buf, int off, int len)
  - количество байт
  - -1 (конец потока)
- int read()
  - read(buf, 0, 1)

00000000	00000000	'ъ'

11111111	11111111	11111111	11111111

- int available()
- long skip(long n)
- void close()
- boolean markSupported()
- void mark(int limit)
- void reset()

java.io.OutputStream

- abstract void write(**int** b)
  - записываемый байт

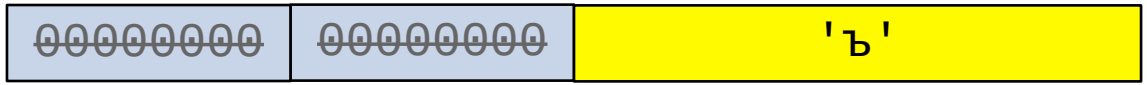
~~00000000~~	~~00000000~~	~~00000000~~	10100111

- void write(byte[] buf, int off, int len)

- void flush()
- void close()

java.io.Writer

- abstract write(char[] buf, int off, int len)
- void write(int c)
  - write(buf, 0, 1)



- void flush()
- void close()

- Writer append(int c)
- Writer append(CharSequence cs)

Interface Flushable

- flush() - очистка внутреннего кэша или буфера
- данные сливаются в приемник
  - После выполнения метода внутренние буферы и кэши должны быть пустыми, данные переданы операционной системе для записи

Интерфейс Closeable

- close() throws IOException - освобождение ресурса

```
try {
    InputStream ins = new InputStream();
    ins.read();
} finally {
    if (ins != null) ins.close();
}
```

Интерфейс AutoCloseable

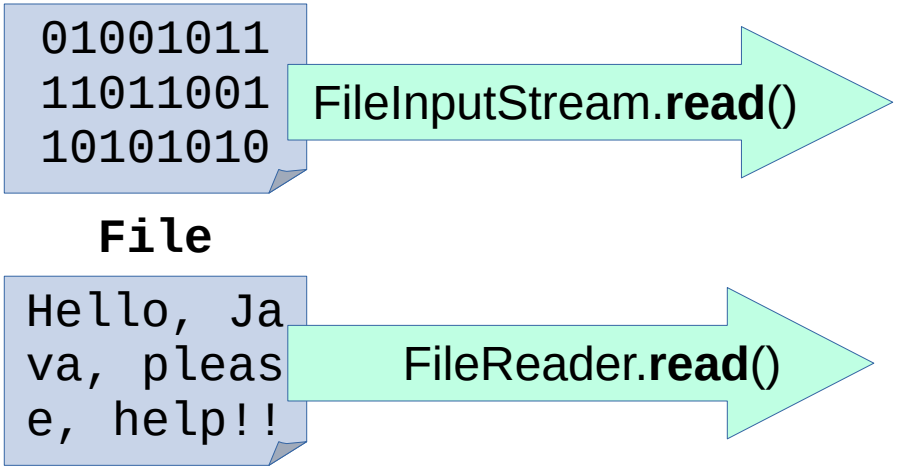
- Closable extends AutoCloseable
- close() вызывается автоматически
- блок try с ресурсом

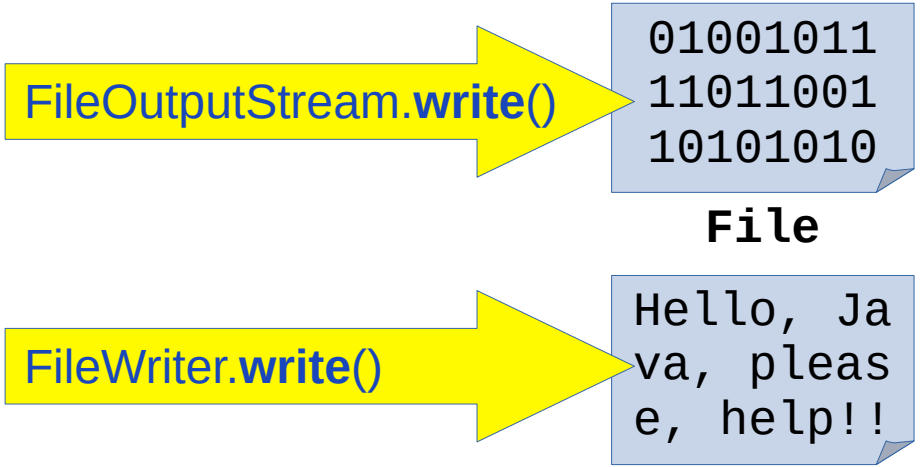
```
InputStream ins = new InputStream();
try(ins) {
    ins.read();
}
```

Специализированные потоки - File

- FileOutputStream
- FileWriter

- FileInputStream
- FileReader





Чтение и запись с помощью потоков

ИТМО

Java 9

```
FileInputStream in = new FileInputStream("in.bin");
FileOutputStream out = new FileOutputStream("out.bin");
try (in, out) {
  int c;
  while ((c = in.read()) != -1) {
    out.write(c);
  }
  out.flush();
} catch (IOException e) {
  System.err.println(e);
}
```

Java

Чтение и запись с помощью потоков

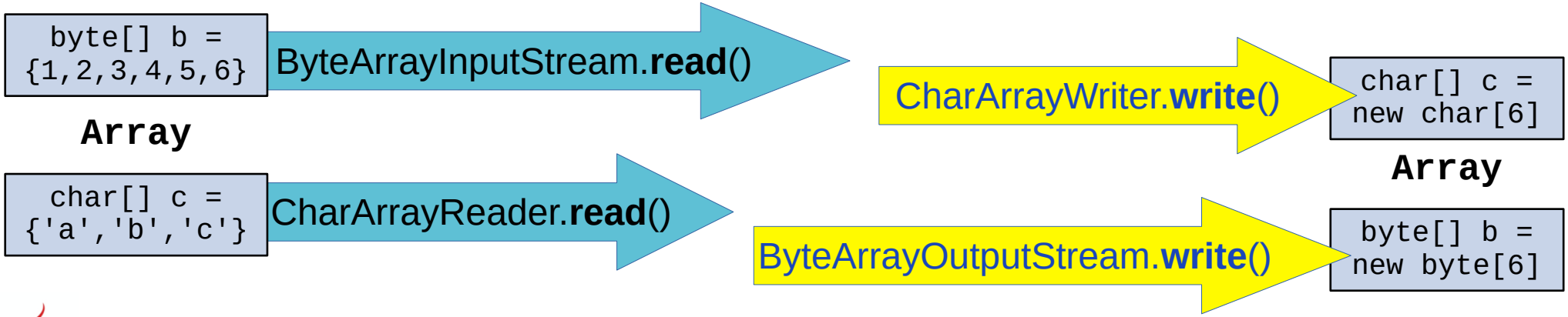
ИТМО

```
try (FileReader in = new FileReader("in.txt");
     FileWriter out = new FileWriter("out.txt")) {
  int c;
  while ((c = in.read()) != -1) {
    out.write(c);
  }
  out.flush();
} catch (IOException e) {
  System.err.println(e);
}
```

Java

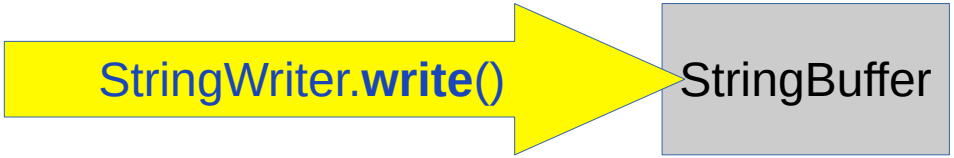
Специализированные потоки - Array

- ByteArrayOutputStream
  - toByteArray()
- CharArrayWriter
  - toCharArray()
- ByteArrayInputStream
- CharArrayReader



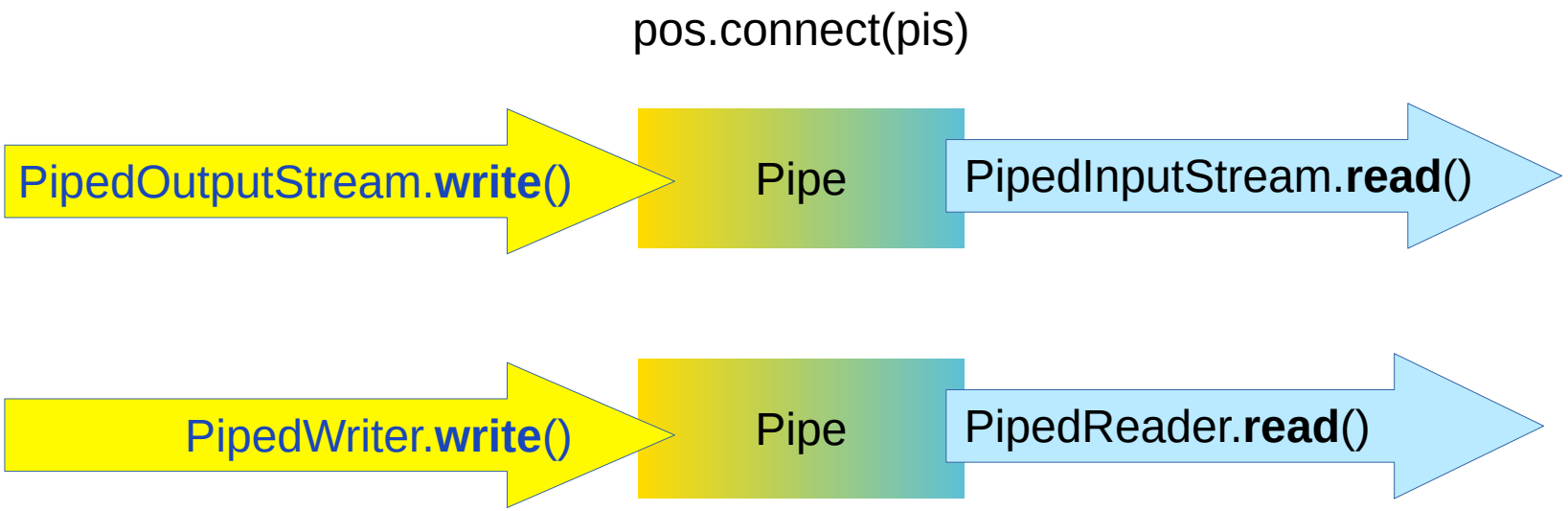
Специализированные потоки - String

- StringWriter
  - StringBuffer getBuffer()
  - String toString()
- StringReader



Специализированные потоки - Pipe

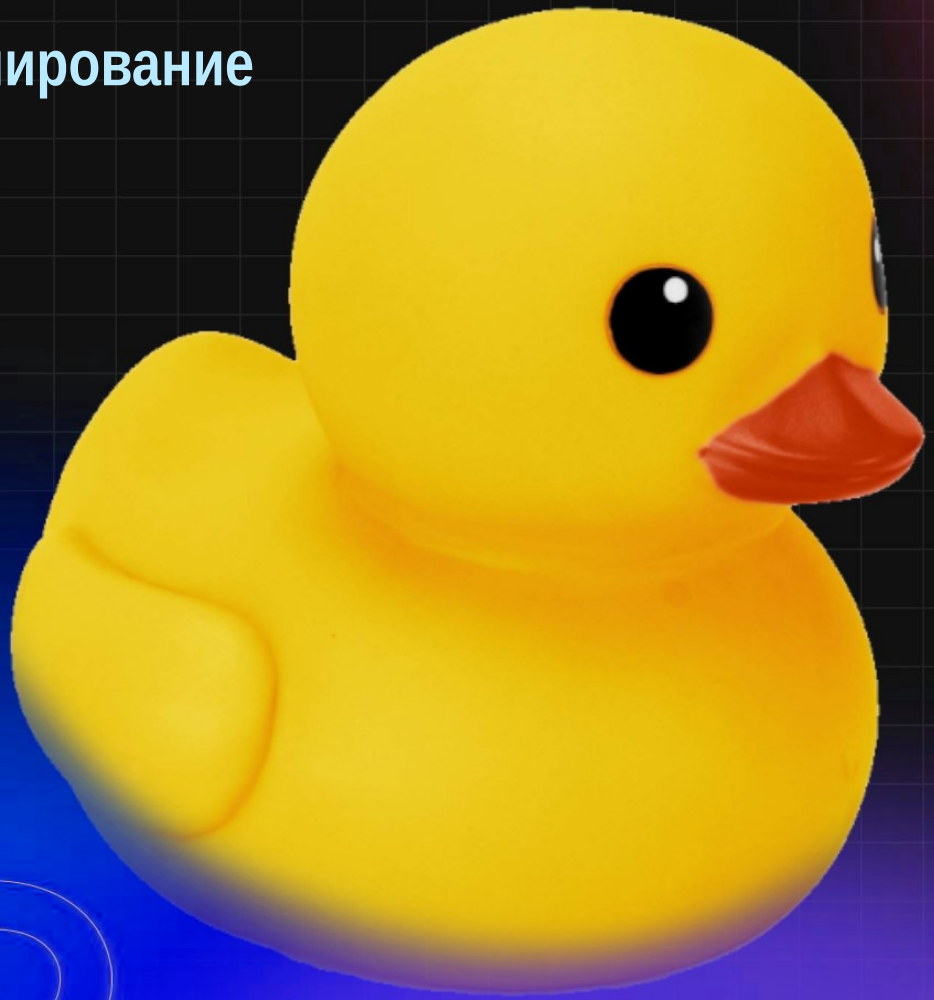
- PipedOutputStream
- PipedWriter
- PipedInputStream
- PipedReader



Программирование

2 семестр

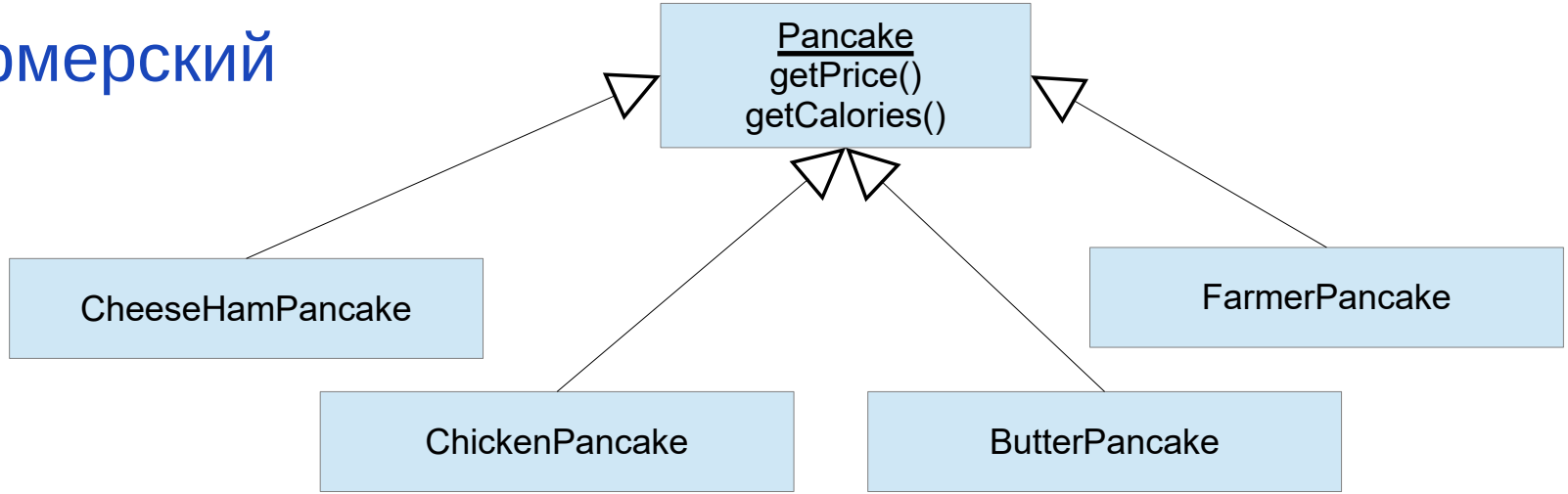
2026



Шаблоны проектирования (продолжение)

Шаблон Decorator - Теремок

- Блин с ветчиной и сыром
- Блин с куриной грудкой
- Блин с маслом
- Блин Фермерский



Шаблон Decorator - Теремок

- Блин с ветчиной и сыром
- Блин с куриной грудкой
- Блин с маслом
- Блин Фермерский

- Добавки
  - Огурцы
  - Лук-фри
  - Картофельное пюре
  - Сметана

- ChickenPancakeWithPicklesAndMashedPotatoes
- FarmerPancakeWithSourCreamAndOnionAndPickles

Шаблон Decorator - Теремок

ИТМО

- Блин с ветчиной и сыром
- Блин с куриной грудкой
- Блин с маслом
- Блин Фермерский

- Добавки
  - Огурцы 30 руб.
  - Лук-фри
  - Картофельное пюре
  - Сметана

```
class PicklePancake extends Pancake {
  Pancake base;
  PicklePancake(Pancake p) { base = p; }
  double getPrice() { return base.getPrice() + 30); }
}
```

Шаблон Decorator - Теремок

- Блин с ветчиной и сыром
- Блин с куриной грудкой
- Блин с маслом
- Блин Фермерский

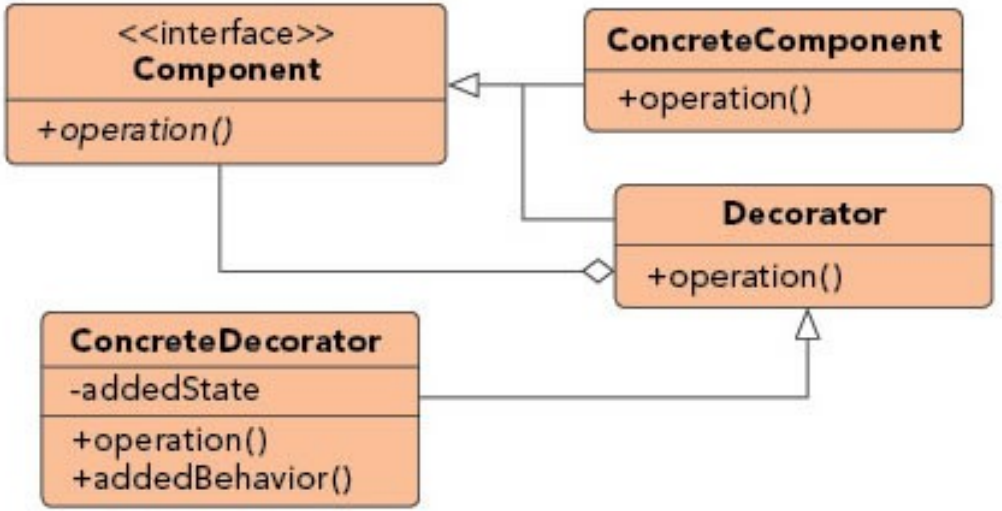
- Огурцы
- Лук-фри
- Картофельное пюре
- Сметана

```
Pancake p =
    new MashedPotatoPancake(
        new PicklePancake(
            new CheeseHamPancake()));

p.getPrice();
```

Шаблон Decorator

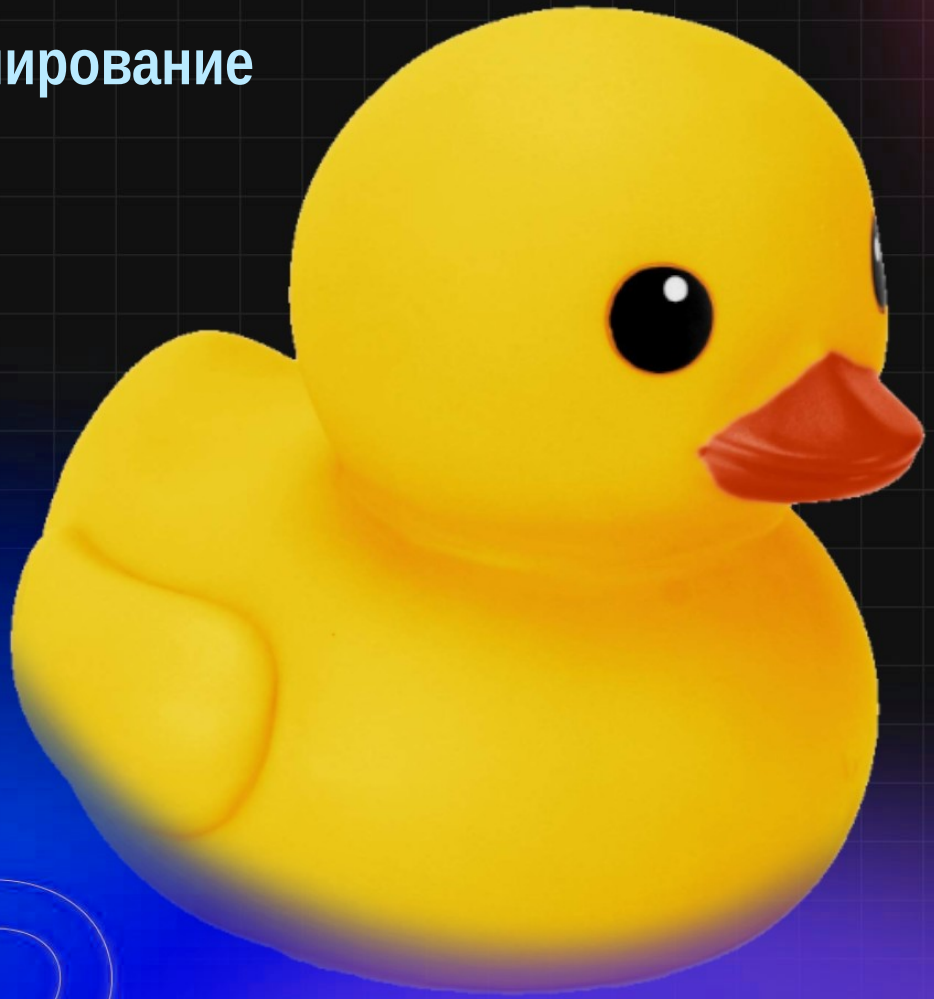
- Позволяет добавлять функциональность динамически
- Вместо большой иерархии - несколько декораторов
- Сложность конфигурирования



Программирование

2 семестр

2026



Ввод-вывод (продолжение)

Потоки-фильтры - декораторы

- FilterInputStream(InputStream)
- FilterOutpurStream(OutpurStream)
- FilterReader(Reader)
- FilterWriter(Writer)
  - исходный поток = аргумент конструктора
  - поток-фильтр = декоратор

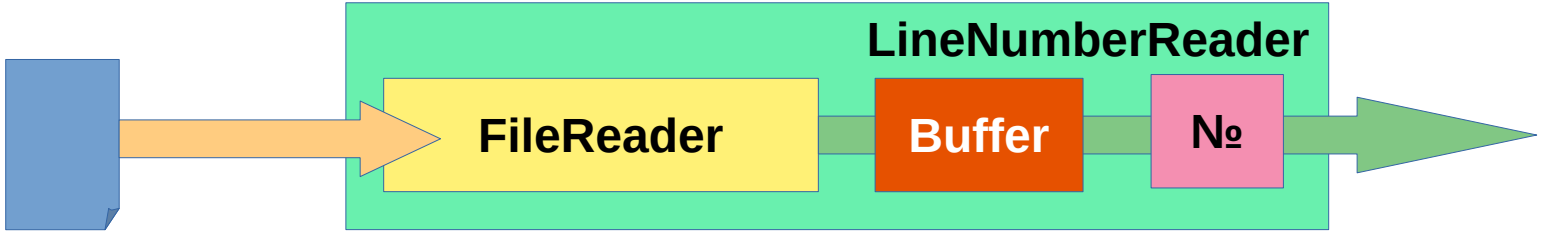
Потоки-фильтры - Buffered

- BufferedInputStream
- BufferedReader
- BufferedOutpurStream
- BufferedWriter

- Буфер для повышения производительности
- Возможность построчной работы

Потоки-фильтры - LineNumber

- ~~LineNumberInputStream~~
- LineNumberReader extends BufferedReader
  - getLineNumber()
  - setLineNumber(int) - меняет только номер, не саму строку



Чтение и запись с помощью потоков

ИТМО

```
try (LineNumberReader in = new LineNumberReader(
                                new FileReader("in.txt"));
     BufferedWriter out = new BufferedWriter(
                                new FileWriter("out.txt")) {
  String line;
  while ((line = in.readLine()) != null) {
    out.write(in.getLineNumber() + ": " + line);
    out.newLine();
  }
  out.flush();
} catch (IOException e) {
  System.err.println(e);
}
```

Java

☑ Потоки-фильтры - байт ↔ символ

ИТМО

- Чтение-запись в других кодировках
- InputStreamReader extends Reader
  - байты в символы
  - new InputStreamReader(InputStream in, *кодировка*)
- OutputStreamWriter extends Writer
  - символы в байты
  - new OutputStreamWriter(OutputStream out, *кодировка*)

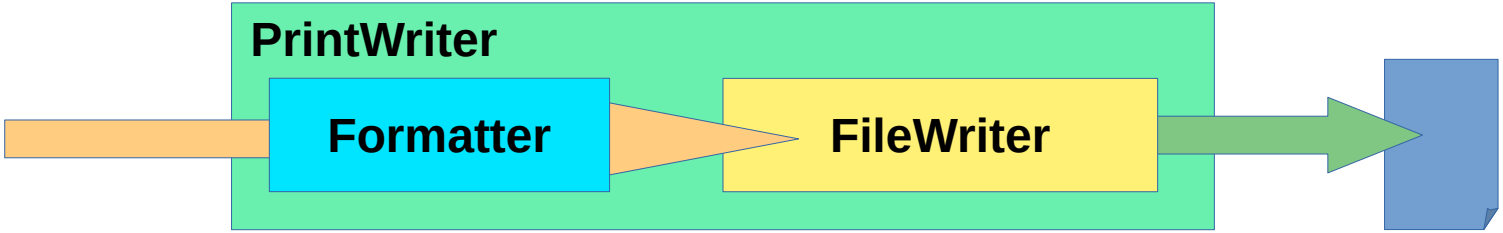
Java

PrintStream, PrintWriter

- PrintStream, PrintWriter
- print, println — один аргумент
- printf, format — форматная строка + аргументы

```
format("%c = %2$+9.7f", 'π', Math.PI);

π = +3,1415927
```



PrintStream, PrintWriter

ИТМО

- %[индекс$][флаги][размер][.точность]формат
- %b - boolean
- %h - hashcode
- %t = time/date
- %% - %
- %n - newline
- %d - decimal
- %o - octal
- %h - hex
- %s - string
- %c - char
- %f - float
- %e - exponent
- %g - %e / %f
- %a - hex float

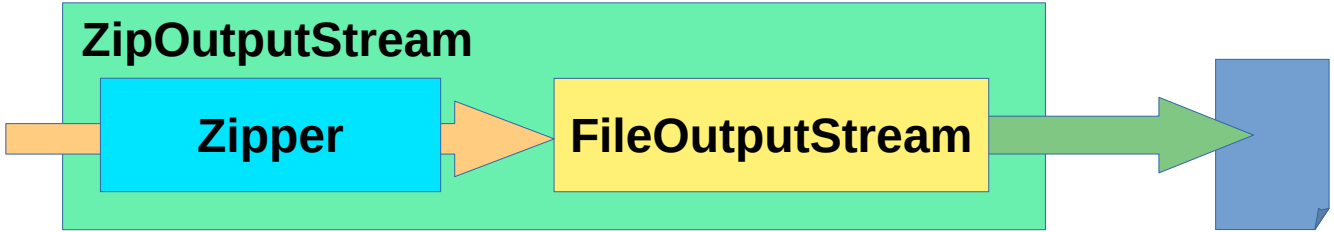
- индекс – номер аргумента
- флаги – зависят от формата
- размер – количество символов
- точность – после запятой

```
format("%c = %2$+9.7f", 'π', Math.PI);

π = +3,1415927
```

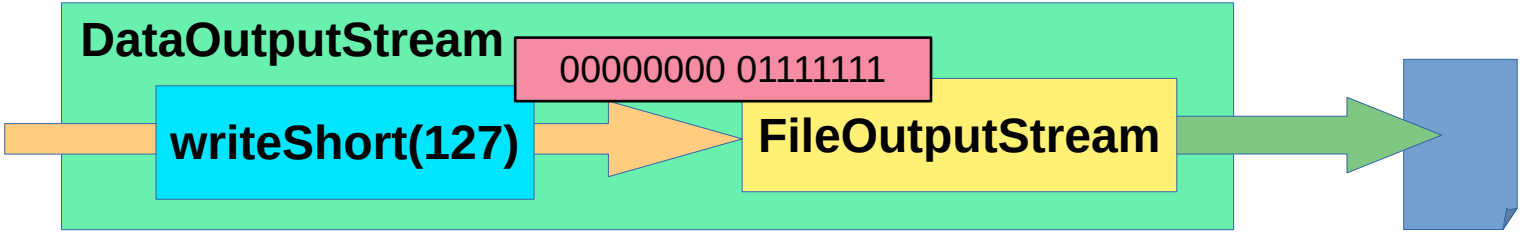
Потоки-фильтры - Inflater/Deflater

- потоки с компрессией
- InflaterInputStream
  - GzipInputStream
  - ZipInputStream
    - JarInputStream
- DeflaterOutputStream
  - GzipOutputStream
  - ZipOutputStream
    - JarOutputStream



DataInputStream, DataOutputStream

- DataInputStream, DataOutputStream
- примитивы и строки ↔ байты
- Последовательность при чтении - такая же как при записи:
- DataOutputStream.writeInt(int), DataInputStream.readInt()
  - byte, short, long, float, double, char, boolean
  - writeUTF(String), readUTF()



Сериализация объектов

ИТМО

- Сериализация — запись объектов в виде потока байтов
- Классы ObjectOutputStream, ObjectInputStream
- Интерфейс-метка Serializable
- При записи объекты записываются с порядковым номером (serial number)
- Объекты записываются в поток иерархически (deep copy)
- Объект записывается в поток только один раз, потом используется ссылка на номер объекта
- При чтении одного и того же объекта из одного потока, он восстанавливается один раз
- При чтении объекта из двух потоков, объект восстанавливается дважды

Java

ObjectOutputStream, ObjectInputStream

ИТМО

- Те же методы, что и у DataOutputStream, DataInputStream
  - + writeObject(Object)
  - + Object readObject()
- Методы записывают/читают:
  - класс объекта
  - сигнатуру класса (зависит от имени и модификаторов класса, интерфейсов, типов и имен полей, имен и сигнатур конструкторов и методов)
  - сигнатура класса может быть записана в поле serialVersionUID
  - значения нестатических и нетранзитивных полей данного класса, и всех его суперклассов
  - поля с модификаторами static и transient не сериализуются

Java

Проблемы стандартной сериализации

ИТМО

- Сериализованный объект зависит от внутренней структуры класса
- Объекты могут создаваться в обход конструкторов
- Потенциальные проблемы совместимости версий
- Возможные проблемы с:
  - продолжительностью сериализации
  - необходимой памятью на сериализацию
  - необходимой глубиной стека

Java

Замена стандартной сериализации

ИТМО

- Задать несериализуемым полям модификатор transient
- Реализовать методы в сериализуемом классе
  - private void writeObject(ObjectOutputStream os)
  - private void readObject(ObjectInputStream is)
- внутри этих методов вызываются методы
  - os.defaultWriteObject()
  - is.defaultReadObject()
- Суперкласс не трогаем

Java

Замена стандартной сериализации

ИТМО

- Реализуем интерфейс Externalizable
- реализуем методы (полный контроль сериализации)
  - writeExternal(ObjectOutput o)
  - readExternal(ObjectInput i)
- Необходимо самостоятельно обрабатывать суперкласс
- При сериализации вместо стандартного механизма будет вызван метод writeExternal, при десериализации - readExternal

Java

Консольный ввод-вывод

- Стандартные потоки ввода-вывода
  - InputStream System.in
  - PrintStream System.out
  - PrintStream System.err
- java.io.Console
  - Console c = System.console()
  - cons.readLine()
  - cons.readPassword()
  - cons.printf()
  - cons.format()

Scanner

- Конструкторы
  - Scanner(File)
  - Scanner(Path)
  - Scanner(InputStream)
  - Scanner(Readable)
  - Scanner(String)
- Методы
  - boolean hasNext(), String next()
  - boolean hasNextInt(), nextInt()
    - ….long, byte, short, float, double, boolean, char
  - String nextLine(), nextPattern()
  - void useDelimiter(String)
  - void useRadix(int)

Scanner

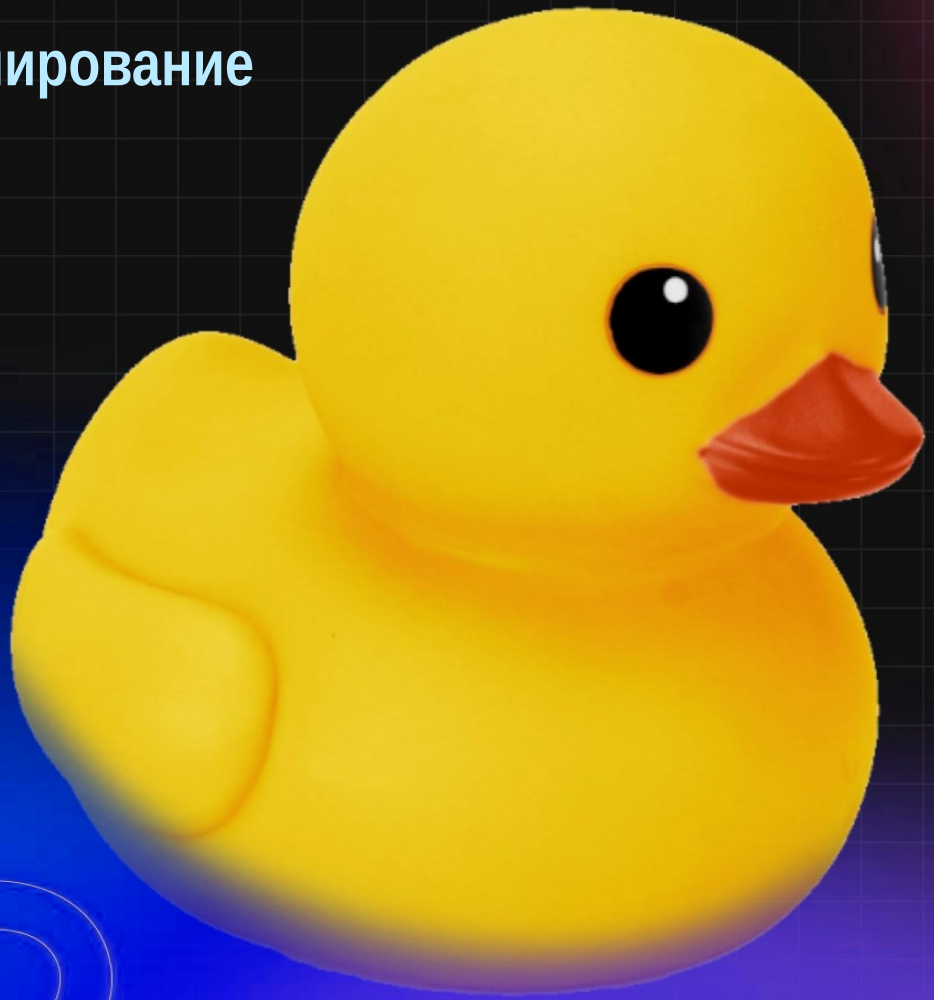
```
Scanner scanner = new Scanner(System.in);
List<List<Integer>> ints = new ArrayList<>();
List<Integer> intList = null;
while (scanner.hasNext()) {
    String line = scanner.nextLine();
    Scanner intScanner = new Scanner(line);
    intList = new ArrayList<Integer>();
    while (intScanner.hasNextInt()) {
        intList.add(intScanner.nextInt());
    }
    ints.add(intList);
    intScanner.close();
}
```

```
100 200 300 400 500
1 2 3 4 5 6 7 8 9
-1 -2 -3 -4 -5
9 8 7 6 5 4 3 2 1
45 0 -45
```

Программирование

2 семестр

2026



Вспомогательные классы

Класс java.util.Random

- public Random()
- public Random(long seed)
- nextInt(), nextDouble, nextLong, nextGaussian
- IntStream ints()
- LongStream longs()
- DoubleStream doubles()

Запись и чтение CSV, JSON, XML

- Запись
  - println() / format()
- Чтение
  - Scanner, StreamTokenizer, Regex
- CSV
  - ! специальные символы, разделители, переносы строк

Примеры форматов

ИТМО

```
public class User {
  int id;
  java.lang.String name;
  public User(int i, String n) {
    id = i; name = n;
  }
}
User u = new User(1, "Ivan");
```

- Java serialization

```
AC ED 00 05 73 72 00 14 52 45
50 4C 2E 24 4A 53 68 65 6C 6C
24 32 30 24 55 73 65 72 D5 FB
60 F0 BA 3D B2 CD 02 00 02 49
00 02 69 64 4C 00 04 6E 61 6D
65 74 00 12 4C 6A 61 76 61 2F
6C 61 6E 67 2F 53 74 72 69 6E
67 3B 78 70 00 00 00 01 74 00
04 49 76 61 6E
```

- CSV

```
id, name
1,"Ivan"
```

- JSON

```
{
  "id":1,
  "name":"Ivan"
}
```

- XML

```
<User>
  <id>1</id>
  <name>Ivan</name>
</User>
```

Запись и чтение JSON, XML

- Binding
  - XML/JSON <-> Java Object (целиком)
  - низкая скорость, настройка на класс, много памяти, простая обработка
  - JAXB
- DOM (Document Object Model)
  - XML/JSON <-> Object Tree (Graph)
  - Node, Element
  - универсальная модель, много памяти, средняя сложность
  - XML DOM
- Event/Stream
  - element start / element end
  - высокая скорость, мало памяти, сложнее обработка
  - SAX, StAX

Запись и чтение CSV, JSON, XML

- Стандартные библиотеки java
  - XML
    - модуль java.xml - переместился в Jacarta EE
  - JSON
    - javax.json.* (Java EE)

Запись и чтение CSV, JSON, XML

ИТМО

- Сторонние библиотеки
  - CSV
    - OpenCSV
    - Apache Commons CSV
  - JSON
    - GSON
    - Jackson
  - XML
    - XStream
    - JacksonXML
    - Jakarta XML Bind

```
var writer = new CSVWriter(new FileWriter(file));
String[] line = {""+user.id, user.name});
writer.writeNext(line);
var reader = new CSVReader(new FileReader(file));
String[] values = reader.readNext();
```

```
Gson gson = new Gson();
String json = gson.toJson(user);
User user = gson.fromJson(json, User.class);
```

```
XStream xstream = new XStream();
String xml = xstream.toXML(user);
User user = xstream.fromXML(xml, User.class);
```

Java

Javadoc

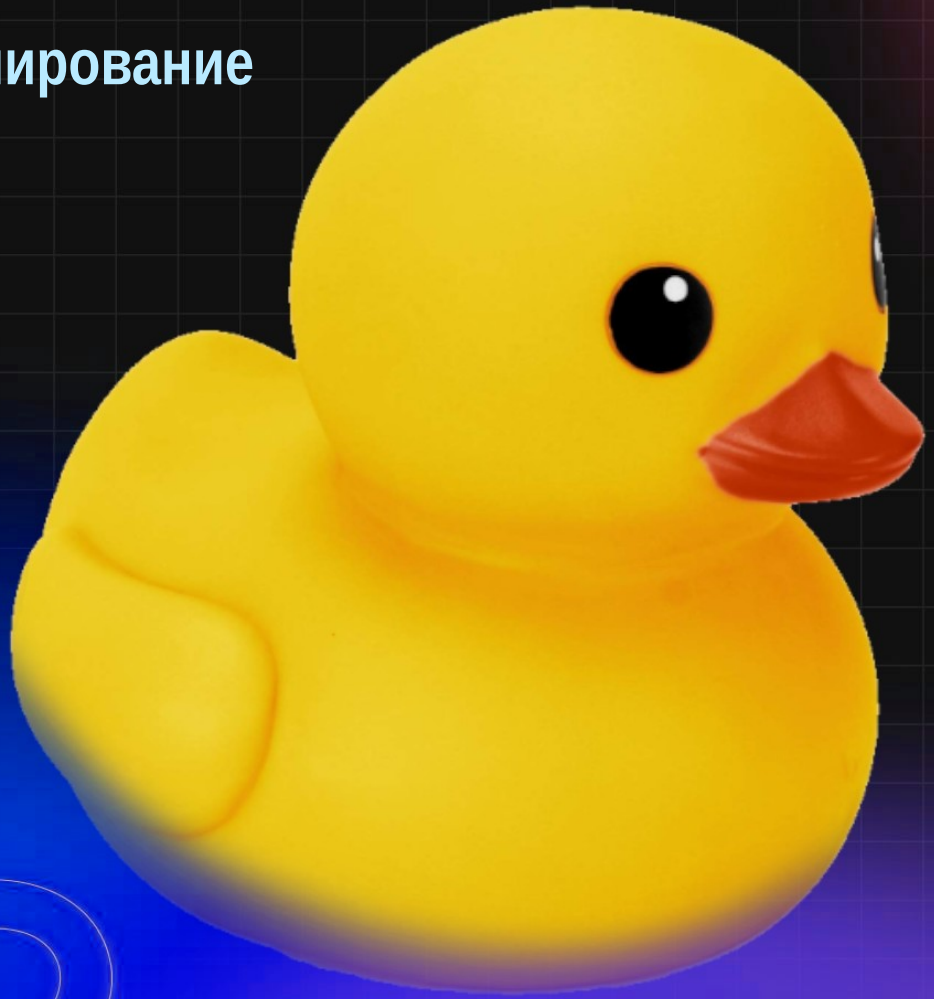
```
/**
 * This is my class
 * @author I
 */
public class MyClass {
/**
 * This is my method
 * @params x just x
 * @return double x
 */
  public int doubleX(int x) {
    return x * 2;
  }
}

javadoc -d doc *.java
```

Программирование

2 семестр

2026



Файлы (IO / NIO.2)

Файлы и файловые системы

- Файловая система
  - Каталоги
  - Файлы
    - Жесткие ссылки
    - Символические ссылки
- Путь к файлу
  - абсолютный (от корня)
  - относительный
- Windows (NTFS)
  - раздельные FS
  - разделитель - \
  - C:\Users\student\file.txt
- Linux / UNIX / MacOS
  - виртуальная FS
  - разделитель - /
  - /home/student/file.txt

Файлы

- Класс **java.io.File** - все операции с путями и файлами
- Интерфейс **java.nio.file.Path**
  - символические ссылки и расширенные атрибуты
  - альтернативные файловые системы
  - копирование и перемещение файлов
  - методы бросают исключения
  - настраиваемый разделитель

- Преобразование между File и Path
  - `File Path.toFile()`
  - `Path File.toPath()`

Файловые системы

- abstract class java.nio.file.FileSystem — файловая система
  - Методы:

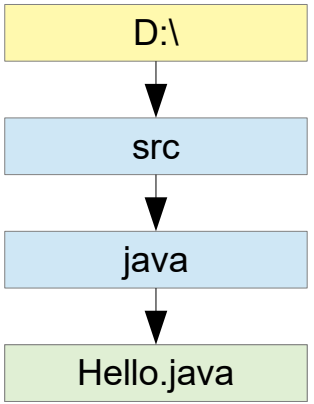
```
Iterable<Path> getRootDirectories()
Path getPath(String first, String... more)
String getSeparator()
```

- класс-утилита java.nio.file.FileSystems

```
FileSystem getDefault()
```

Пути к файлам

- интерфейс java.nio.file.Path — путь к файлу
  - абсолютный / относительный
  - пустой — текущая директория
- класс-утилита java.nio.file.Paths




```
Path p = Paths.get("java","Hello.java");
```

Действия с файлами: проверки

- Класс-утилита java.nio.file.Files

```
boolean Files.exists(Path)
boolean Files.notExists(Path)
boolean Files.isReadable(Path)
boolean Files.isWritable(Path)
boolean Files.isExecutable(Path)
```

Работа с файлами: create/delete

```
Path Files.createFile(Path)
Path Files.createTempFile(String prefix, String suffix)

void Files.delete(Path)
boolean Files.deleteIfExists(Path)
```

Работа с файлами: чтение и запись

```
byte[] Files.readAllBytes(Path)
List<String> Files.readAllLines(Path)

Files.write(Path, byte[])
Files.write(Path, Iterable<CharSequence>)
```

Работа с файлами: чтение и запись

ИТМО

```
FileInputStream / FileReader / FileOutputStream / FileWriter
new BufferedXXX(new FileXXX(File/String))
```

java.io

```
Files.newInputStream(Path, OpenOptions...)
Files.newOutputStream(Path, OpenOptions...)
Files.newBufferedReader(Path, Charset, OpenOptions...)
Files.newBufferedWriter(Path, Charset, OpenOptions...)
```

java.nio

чтение

```
String line;
while ((line = reader.readLine()) != null) { }
```

запись

```
String text;
writer.write(s, 0, s.length());
```

Работа с файлами: чтение и запись

ИТМО

запись

```
var path = Path.of("/home/s000000/prog/my_text.txt");

String content = "Hello all!";
Files.writeString(path, content);

List<String> lines = List.of("Hello all!", "Bye all!");
Files.write(path, lines);
```

чтение

```
var path = Path.of("/home/s000000/prog/my_text.txt");

String content = Files.readString(path);

List<String> lines = Files.readAllLines(path);
```

Java